Q: 几个问题:

- 一般ai agent 如 claude code cli 中使用/ 可以呼出的命令是通过 commands 还是 skills 定义的？我想在当前开发的 KCA 平台提供类似的 CLI , 给Claude Code 的提示词应该如何描述?
- 原先打算使用Lovable 开发product UI shell, 现在感觉 Claude Code 也可以胜任，特别是当平台后端， code competition agent 和 supabase migration 都在 Claude Code 中开发时，继续在 Claude Code 中开发更顺畅，你觉得呢?
- 开发时想尽量放宽权限，除了 rm, drop database 等不可逆操作外其他都允许，不需要 ask，这样 Claude Code 跑 /goal 执行 milestone tasks 时除了 decision making 的问题其他不需要等 human approval; 产品的 code competition agent 也类似，除了 direction/decision 问题其他大部分操作都允许执行，sources 中已经包含当前 .claude 目录下相关配置文件，请建议并提供权限宽松的版本

根据 2026-05-20 上传的 `.claude` 配置和最新 Claude Code 官方文档，我的建议是：

1. **Claude Code 里的自定义 `/xxx` 现在优先按 skills 设计，不再优先按 legacy commands 设计。**
2. **KCA 平台自己的 CLI 应该分成：deterministic CLI commands + agent-facing skills/workflows + executable tools + permission policy。**
3. **Product UI shell 继续在 Claude Code 里开发会更顺畅；Lovable 可以留作视觉原型/marketing shell，不建议作为复杂后端+agent+Supabase 的 source of truth。**
4. **权限可以明显放宽，但不要用裸 `bypassPermissions`；更推荐 "allow everything useful + deny/ask irreversible operations + hook audit"。**

1. `/` 命令到底是 commands 还是 skills?

现在 Claude Code 里有三类 `/` 入口：

类型	例子	本质	你在 KCA 里该怎么类比
Built-in commands	`/model`，`/permissions`，`/compact`，`/diff`，`/goal`	Claude Code CLI 内置逻辑	KCA 的固定 CLI 子命令，比如 `kca job run`、`kca decision approve`
Bundled skills	`/batch`，`/debug`，`/loop`，`/run`，`/verify`，`/simplify`	prompt-based workflow，让 Claude 用工具完成任务	KCA 的 agent workflow，比如 `/triage-log`、`/run-benchmark`、`/prepare-submission`
Custom skills / legacy commands	`.claude/skills/foo/SKILL.md` 或 `.claude/commands/foo.md`	自定义 slash command	KCA 应优先实现 `.kca/skills/<name>/SKILL.md` 风格

Claude Code 官方文档现在明确说：skills 可以通过 `SKILL.md` 创建，Claude 会在相关时自动使用，也可以用 `/skill-name` 直接调用；而且 **custom commands 已经 merged into skills**，`.claude/commands/deploy.md` 和 `.claude/skills/deploy/SKILL.md` 都会创建 `/deploy`，旧 commands 仍兼容。(Claude API Docs)

所以给 Claude Code 的开发提示词里，不要说"实现 Claude Code commands clone"。更准确应该说：

```
Implement KCA's command/skill/tool architecture inspired by Claude Code:

- CLI commands are deterministic user/operator entry points.
- Skills are prompt-based agent workflows with metadata, arguments, allowed tools, and approval policy.
- Tools are executable actions used by agents: shell, file IO, notebook conversion, Kaggle API, Supabase API, LiteL
- Permissions are enforced outside the model by policy and hooks, not just by prompt instructions.
```

Claude Code 的 command reference 也说明， `/` 里面既有 built-in commands，也有标记为 Skill 的 bundled skills；标记为 Skill 的条目使用和自定义 skills 相同的机制，而其他条目是 CLI 内置行为。(Claude)

2. KCA 平台自己的 CLI 应该如何设计?

我建议 KCA 不要直接把所有东西叫 “commands”。用四层命名更清楚：

```
Command:
  用户/平台调用的确定性 CLI/API 入口。

Skill:
  agent-facing workflow recipe，可被 slash 调用，也可被 agent 自动选择。

Tool:
  可执行动作，带输入输出 schema、权限、审计、超时、sandbox。

Job:
  持久化任务记录，包含 prompt、模型、状态、日志、预算、artifact、decision gates。
```

推荐目录结构

```
.kca/
  skills/
    run-milestone/
      SKILL.md
    triage-log/
      SKILL.md
    prepare-submission/
      SKILL.md
    review-pr/
      SKILL.md
  tools/
    tools.yaml
  permissions/
    default.yaml
    permissive-dev.yaml
    competition-runner.yaml

tools/kca/
  kca.py
  commands/
    job.py
    skill.py
    tool.py
    decision.py
    workspace.py
    kaggle.py
    supabase.py
  agent/
    runner.py
    skill_loader.py
    tool_executor.py
    permission_policy.py
    audit.py
```

KCA CLI 命令建议

```
kca job create --skill run-milestone --prompt docs/tasks/M001.md
kca job run <job-id>
kca job logs <job-id>
kca job attach <job-id>
kca job cancel <job-id>

kca skill list
kca skill show run-milestone
kca skill run run-milestone docs/tasks/M001.md

kca tool list
kca tool doctor

kca decision list
kca decision approve <decision-id>
kca decision reject <decision-id>

kca workspace create --job <job-id>
kca workspace clean --job <job-id>

kca kaggle fetch <competition>
kca kaggle watch <competition>
kca kaggle submit <run-id>   # decision-gated

kca supabase migrate local
kca supabase migrate remote  # decision-gated
```

KCA skill 示例

```
---
name: run-milestone
description: Execute a milestone task end-to-end in an isolated workspace until acceptance criteria pass or blocker
argument-hint: "[task-md-path]"
arguments:
  - task_path
allowed-tools:
  - read_file
  - write_file
  - edit_file
  - bash
  - git
  - pytest
  - npm
  - supabase_local
  - litellm
approval-policy: permissive-dev
disable-model-invocation: true
---

Run milestone task `$task_path`.

Workflow:
1. Read the task file and CLAUDE.md.
2. Inspect the current architecture.
3. Create or use an isolated workspace/branch.
4. Implement the smallest coherent change.
5. Run relevant tests, lint, type checks, and smoke tests.
6. Iterate until acceptance criteria pass.
7. Produce:
   - summary
   - changed files
   - tests run
   - risks
   - unresolved blockers
   - next recommended task
8. If a decision gate is required, stop and create a decision record instead of proceeding.

Stop conditions:
- Acceptance criteria pass.
- A decision gate is reached.
- Max turns/budget/time is reached.
```

Claude Code skills 支持 YAML frontmatter、`description`、`arguments`、`allowed-tools`、`disable-model-invocation`、`context: fork` 等字段；`allowed-tools` 可以让 skill active 时工具免确认，但不能限制其他工具，真正限制仍要靠权限设置/deny rules。(Claude API Docs)

3. 给 Claude Code 的实现提示词

可以直接发给 Claude Code：

```
Read CLAUDE.md and current KCA architecture. Implement a KCA command/skill/tool architecture inspired by Claude Cod

Definitions:
- Commands are deterministic CLI/API entry points.
- Skills are agent-facing workflow recipes stored under .kca/skills/<skill-name>/SKILL.md.
- Tools are executable actions with schemas, timeouts, audit logging, permission labels, and sandbox metadata.
- Jobs are durable records that run skills through an agent loop.

Tasks:
1. Add a skill loader that reads .kca/skills/<name>/SKILL.md with YAML frontmatter.
2. Add a command registry for KCA CLI subcommands.
3. Add a tool registry with tool metadata and approval policy labels.
4. Add a permission policy layer with allow/ask/deny decisions.
5. Add slash-style invocation support, e.g. kca skill run run-milestone docs/tasks/M001.md.
6. Add audit logging for tool execution.
7. Add tests for:
   - skill parsing
   - argument substitution
   - permission policy resolution
   - command registry lookup
   - decision gates
8. Do not hardcode secrets.
9. Do not call real Kaggle submission or remote Supabase production operations in tests.

Acceptance criteria:
- `kca skill list` shows available skills.
- `kca skill show <name>` prints parsed metadata and instructions.
- `kca skill run <name> ... --dry-run` shows planned tool permissions and rendered prompt.
- Unit tests cover skill parsing, command registry, and permission policy.
- Documentation explains Commands vs Skills vs Tools vs Jobs.
```

4. UI shell：继续用 Claude Code 还是 Lovable?

我现在更倾向于：**MVP 阶段继续在 Claude Code 里开发 UI shell**。

原因是你的产品不是纯 landing page 或 CRUD demo，而是一个复杂系统：

```
Product UI shell
  + Supabase schema / migrations
  + code competition job queue
  + agent runs
  + local/cloud model routing
  + logs / artifacts
  + decisions / approvals
  + PR / GitHub / Kaggle integration
```

这些都在同一个 repo、同一套 migrations、同一套 tests、同一套 agent workflow 里时，Claude Code 的 repo-level context、terminal、tests、migration、diff review 会比 Lovable 更稳。你之前也已经把方向定为 Claude Code/CLI 负责开发、Agent SDK/队列负责产品化、任务通过 `CLAUDE.md` 和 `docs/tasks/*.md` 管理；这和现在继续在 Claude Code 开发 UI shell 是一致的。

Lovable 可以保留在这些场景：

- 快速探索 dashboard visual style
- 生成 marketing page / onboarding mockup
- 做几个 UI flow 的可视化原型
- 给 Claude Code 提供设计参考

但我不建议让 Lovable 成为 source of truth，尤其不要让它单独维护 Supabase migrations、agent job schema、runner backend。你的 source of truth 应该是 GitHub repo + Claude Code + CI + Supabase migrations。

5. 你当前 `.claude` 权限配置的问题

你当前 `.claude/settings.json` 已经很宽：它允许 `Read`、`Edit`、`Write`、`MultiEdit`、`WebFetch`、`WebSearch`、`Bash`，还允许 `git push`、`supabase db push`、`docker`、`pytest`、`npm test`、KCA CLI 等大量命令。

真正会让 `/goal` 停下来等你的，是两层：

第一层是 `settings.json` 的 `ask`：

```
"ask": [
  "Bash(gh pr create*)",
  "Bash(gh pr merge*)",
  "Bash(python tools/kca/kca.py kaggle submit*)",
  "Bash(python tools/kca/kca.py kaggle push*)",
  "Bash(python tools/kca/kca.py kaggle upload*)"
]
```

第二层是 `guard_tool.py` 的 `ask_patterns`，它现在会对 `kaggle competitions submit`、`git push`、`gh pr create/merge`、`supabase db push` 等返回 ask。

所以如果你只改 `.claude/settings.json`，但不改 hook，Claude 仍然会被 hook 拦下来。

另外，当前 `.claude/settings.local.json` 设置了：

```
"worktree": {
  "bgIsolation": "none"
}
```

这会让 background jobs 直接编辑当前 working copy。对单 session 方便，但对并行 `/goal` / background tasks 风险比较大；我建议改回 `"worktree"`，除非你确定同一时间只有一个 agent 在当前 checkout 工作。

官方设置文档也说明，project settings 是 `.claude/` 共享给 repo 协作者，local settings 是 `.claude/settings.local.json`、只影响本 repo 的你自己且不共享。(Claude API Docs)

6. 权限模式推荐：不要裸 `bypassPermissions`

Claude Code 官方权限系统里，`allow` 会免确认，`ask` 会要求确认，`deny` 会阻止；规则顺序是 **deny → ask → allow**，所以 deny 永远优先。(Claude)

你想要的模式可以这样表达：

```
允许：
  读写项目文件、编辑代码、运行测试、lint、build、docker、git commit/push、gh pr create、local Supabase migration、大多数 KCA

需要 ask：
  方向性/决策性操作：
  - merge PR
  - Kaggle submit/upload/push
  - remote/prod database migration
  - rm / git clean / destructive cleanup
  - drop/truncate/delete database data
  - production deployment

直接 deny：
  - 读取 secrets
  - 编辑真实 .env
  - sudo
  - rm -rf / 或 home
  - force push
  - privileged docker
  - cloud metadata / credential exfiltration
```

官方也明确提醒： `bypassPermissions` 会跳过几乎所有 permission prompts，包括 `.git` 、 `.claude` 、 `.vscode` 等目录写入；只在容器/VM 等隔离环境中使用。(Claude)

所以我建议：

```
# 长任务推荐
claude --permission-mode dontAsk

# 或者你想保留 auto classifier：
claude --permission-mode auto
```

`dontAsk` 的特点是：未预先 allow 的工具会被自动拒绝，而不是卡住等你；配合非常宽的 allow list 和明确 deny list，适合 unattended `/goal` 。(Claude)

7. 宽松版 `.claude/settings.json`

建议把 repo 里的共享配置改成下面这样。它比你当前版本更少 ask，允许 `gh pr create` 、 `git push` 、local migration、测试、build、docker、KCA CLI 等操作，但保留 secrets/destructive deny。

```
{
  "$schema": "https://json.schemastore.org/claude-code-settings.json",
  "permissions": {
    "allow": [
      "Read",
      "Edit",
      "Write",
      "MultiEdit",
      "Glob",
      "Grep",
      "LS",
      "NotebookRead",
      "NotebookEdit",
      "WebFetch",
      "WebSearch",
      "Bash",

      "Agent(Explore)",
      "Agent(Plan)",

      "Bash(git status*)",
      "Bash(git diff*)",
      "Bash(git log*)",
      "Bash(git branch*)",
      "Bash(git checkout*)",
      "Bash(git switch*)",
      "Bash(git add*)",
      "Bash(git commit*)",
      "Bash(git push*)",
      "Bash(git fetch*)",
      "Bash(git pull*)",
      "Bash(git rebase*)",
      "Bash(git merge*)",
      "Bash(git worktree*)",

      "Bash(gh issue*)",
      "Bash(gh pr view*)",
      "Bash(gh pr list*)",
      "Bash(gh pr diff*)",
      "Bash(gh pr status*)",
      "Bash(gh pr create*)",
      "Bash(gh pr comment*)",
      "Bash(gh api*)",
      "Bash(gh repo view*)",

      "Bash(uv*)",
      "Bash(python*)",
      "Bash(pytest*)",
      "Bash(ruff*)",
      "Bash(pyright*)",
      "Bash(mypy*)",
      "Bash(make*)",

      "Bash(npm*)",
      "Bash(pnpm*)",
      "Bash(npx*)",
```

```
    "Bash(node*)",
    "Bash(vite*)",

    "Bash(docker*)",
    "Bash(docker compose*)",

    "Bash(supabase status*)",
    "Bash(supabase start*)",
    "Bash(supabase stop*)",
    "Bash(supabase migration*)",
    "Bash(supabase migration up*)",
    "Bash(supabase db diff*)",
    "Bash(supabase gen types*)",

    "Bash(psql*)",
    "Bash(sqlite3*)",

    "Bash(mkdir*)",
    "Bash(touch*)",
    "Bash(cp*)",
    "Bash(mv*)",
    "Bash(chmod +x*)",

    "Bash(python tools/kca/kca.py*)",
    "Bash(python scripts/run_job.py*)",
    "Bash(scripts/*)"
  ],
  "ask": [
    "Bash(rm *)",
    "Bash(rmdir *)",
    "Bash(git clean*)",
    "Bash(gh pr merge*)",
    "Bash(supabase db push*)",
    "Bash(supabase db reset*)",
    "Bash(kaggle competitions submit*)",
    "Bash(kaggle kernels push*)",
    "Bash(kaggle datasets create*)",
    "Bash(kaggle datasets version*)",
    "Bash(python tools/kca/kca.py kaggle submit*)",
    "Bash(python tools/kca/kca.py kaggle push*)",
    "Bash(python tools/kca/kca.py kaggle upload*)"
  ],
  "deny": [
    "Read(./.env)",
    "Read(./.env.local)",
    "Read(./.env.development)",
    "Read(./.env.production)",
    "Read(./.env.test)",
    "Read(./kaggle.json)",
    "Read(./.kaggle/kaggle.json)",
    "Read(~/.kaggle/kaggle.json)",
    "Read(./secrets/**)",
    "Read(./**/kaggle.json)",
    "Read(~/.ssh/**)",
    "Read(~/.aws/**)",
    "Read(~/.config/gcloud/**)",
    "Read(~/.config/gh/hosts.yml)",
```

```
      "Read(./.git/**)",

      "Edit(./.env)",
      "Edit(./.env.local)",
      "Edit(./.env.development)",
      "Edit(./.env.production)",
      "Edit(./.env.test)",
      "Edit(./secrets/**)",
      "Edit(~/.ssh/**)",
      "Edit(~/.aws/**)",
      "Edit(~/.config/gcloud/**)",

      "Bash(sudo*)",
      "Bash(su *)",
      "Bash(doas*)",
      "Bash(rm -rf / *)",
      "Bash(rm -rf /*)",
      "Bash(rm -rf ~*)",
      "Bash(rm -rf $HOME*)",
      "Bash(git push --force*)",
      "Bash(git push -f*)",
      "Bash(docker run *--privileged*)",
      "Bash(chmod 777*)",
      "Bash(chown * / *)",
      "Bash(mkfs*)",
      "Bash(dd *of=/dev*)",
      "Bash(shutdown*)",
      "Bash(reboot*)",
      "Bash(halt*)",

      "Bash(curl *169.254.169.254*)",
      "Bash(wget *169.254.169.254*)",
      "Bash(curl *metadata.google*)",
      "Bash(wget *metadata.google*)",

      "Bash(psql * -c *drop database*)",
      "Bash(psql * -c *drop schema*)",
      "Bash(psql * -c *truncate*)",
      "Bash(psql * -c *delete from*)"
    ]
  },
  "sandbox": {
    "enabled": true,
    "failIfUnavailable": false,
    "autoAllowBashIfSandboxed": true,
    "allowUnsandboxedCommands": true,
    "filesystem": {
      "denyRead": [
        "~/.ssh",
        "~/.aws",
        "~/.config/gcloud",
        "~/.config/gh/hosts.yml"
      ],
      "denyWrite": [
        "/",
        "/etc",
        "/usr",
```

```
      "/var",
      "~/.ssh",
      "~/.aws",
      "~/.config/gcloud",
      "~/.config/gh"
    ],
    "allowWrite": [
      ".",
      "/tmp"
    ]
  },
  "network": {
    "allowedDomains": [
      "api.anthropic.com",
      "claude.ai",
      "github.com",
      "api.github.com",
      "raw.githubusercontent.com",
      "objects.githubusercontent.com",
      "kaggle.com",
      "*.kaggle.com",
      "supabase.co",
      "*.supabase.co",
      "pypi.org",
      "files.pythonhosted.org",
      "registry.npmjs.org",
      "playwright.dev",
      "localhost",
      "127.0.0.1"
    ],
    "deniedDomains": [
      "169.254.169.254",
      "metadata.google"
    ]
  }
},
"hooks": {
  "PreToolUse": [
    {
      "matcher": "Bash|Read|Edit|Write",
      "hooks": [
        {
          "type": "command",
          "command": "\"$CLAUDE_PROJECT_DIR\"/.claude/hooks/guard_tool.py"
        }
      ]
    }
  ],
  "PostToolUse": [
    {
      "matcher": "Bash|Edit|Write",
      "hooks": [
        {
          "type": "command",
          "command": "\"$CLAUDE_PROJECT_DIR\"/.claude/hooks/audit_event.py"
        }
      ]
```

```
      }
    ],
    "ConfigChange": [
      {
        "matcher": "project_settings|skills",
        "hooks": [
          {
            "type": "command",
            "command": "\"$CLAUDE_PROJECT_DIR\"/.claude/hooks/audit_event.py"
          }
        ]
      }
    ]
  }
}
```

说明：

- 我保留了 `rm` 、 `git clean` 、 `gh pr merge` 、remote Supabase push/reset、Kaggle submit/upload 作为 ask。
- 我允许 `gh pr create` ，因为创建 PR 是可回滚、低风险操作；merge 才是决策点。
- 我把 `Edit(./.env*)` 改成了精确 `.env` 文件，避免误伤 `.env.example` 。
- 我建议保留 hook，因为 hook 是 deterministic safety net。官方也说明 hooks 可以在 tool call 前 block/force prompt/skip prompt；但 hook 不能绕过 deny/ask rules，blocking hook 还会优先于 allow rules。(Claude)

8. `.claude/settings.local.json`

你的当前 local 配置只加了几个 allow，并且把 background worktree isolation 设成了 `none` 。

我建议改成：

```
{
  "permissions": {
    "allow": [
      "WebFetch(domain:ndownloader.figshare.com)",
      "WebFetch(domain:supabase-db)"
    ]
  },
  "worktree": {
    "bgIsolation": "worktree"
  }
}
```

如果你明确只跑单个 foreground Claude session，可以临时用：

```
{
  "worktree": {
    "bgIsolation": "none"
  }
}
```

但做 `/goal` 、background agents、多个 terminal 并发时，还是建议 `"worktree"` 。

9. `~/.claude/settings.json`：让 unattended 更少卡住

官方文档有一个细节：`permissions.defaultMode: "auto"` 在 project/local settings 里会被忽略，防止 repo 自己给自己开启 auto；要设在 `~/.claude/settings.json` 或用 CLI flag。(Claude API Docs)

你可以在 Arch runner 的用户级配置里放：

```
{
  "permissions": {
    "defaultMode": "dontAsk"
  }
}
```

或者启动时显式：

```
claude --permission-mode dontAsk
```

更灵活但稍微更有 prompt 风险：

```
claude --permission-mode auto
```

我的推荐：

```
claude --bg \
  --name "M001" \
  --permission-mode dontAsk \
  --model opus \
  --effort high \
  "/goal Execute docs/tasks/M001.md until acceptance criteria pass or stop after 80 turns and summarize blockers"
```

10. 宽松版 `guard_tool.py`

你当前 hook 会对 `git push`、`gh pr create`、`supabase db push` 直接 ask，这会打断你希望的无人值守开发。

建议改成下面这个版本：允许 `git push` 和 `gh pr create`；只对 merge、Kaggle submit、remote DB destructive/migration、rm/git clean/drop/truncate 等操作 ask 或 deny。

```python
#!/usr/bin/env python3
"""Claude Code PreToolUse guardrail hook for permissive KCA development.

Policy:
- Allow normal development work without prompts.
- Deny secret access and host-destructive operations.
- Ask only for irreversible or decision-level operations.
"""
from __future__ import annotations

import json
import os
import re
import shlex
import sys
from pathlib import Path


def emit(decision: str, reason: str) -> None:
    print(json.dumps({
        "hookSpecificOutput": {
            "hookEventName": "PreToolUse",
            "permissionDecision": decision,
            "permissionDecisionReason": reason,
        }
    }))


def normalize_path(value: str) -> str:
    if not value:
        return ""
    return os.path.expanduser(value).replace("\\", "/")


def path_is_protected(path: str) -> bool:
    p = normalize_path(path)
    protected_fragments = [
        "/.env",
        "/secrets/",
        "/.ssh/",
        "/.aws/",
        "/.config/gcloud/",
        "/.config/gh/hosts.yml",
        "/kaggle.json",
        "/.netrc",
        "/id_rsa",
        "/id_ed25519",
    ]
    protected_names = {
        ".env",
        ".env.local",
        ".env.development",
        ".env.production",
        ".env.test",
        "kaggle.json",
        ".netrc",
```

```
        "id_rsa",
        "id_ed25519",
        "hosts.yml",
    }
    name = Path(p).name
    return name in protected_names or any(fragment in p for fragment in protected_fragments)


def compact(command: str) -> str:
    return " ".join(command.strip().split())


def bash_decision(command: str) -> tuple[str | None, str | None]:
    c = compact(command)
    lc = c.lower()

    # Hard deny: host-destructive, privilege escalation, secret exfiltration.
    deny_patterns = [
        (r"\bsudo\b", "Refusing sudo from the agent."),
        (r"\bsu\s+", "Refusing su from the agent."),
        (r"\bdoas\b", "Refusing doas from the agent."),
        (r"\brm\s+-rf\s+(/|~|\$HOME)(\s|$)", "Refusing recursive deletion of root/home."),
        (r"\brm\s+-rf\s+/\S+", "Refusing recursive deletion of absolute filesystem paths."),
        (r"\bchmod\s+777\b", "Refusing chmod 777."),
        (r"\bchown\s+.*\s+(/|~|\$HOME)\b", "Refusing ownership change on broad paths."),
        (r"\bdd\s+.*\bof=/dev/", "Refusing raw disk write."),
        (r"\bmkfs(\.|\s|$)", "Refusing filesystem formatting command."),
        (r"\bshutdown\b|\breboot\b|\bhalt\b", "Refusing host power command."),
        (r"\bgit\s+push\s+(-f|--force|--force-with-lease)", "Refusing force push."),
        (r"\bdocker\s+run\b.*\s--privileged\b", "Refusing privileged Docker container."),
        (r"\b(cat|less|more|head|tail|grep|rg)\b.*(\.env|kaggle\.json|id_rsa|id_ed25519|\.aws|\.ssh|hosts\.yml)", "
        (r"\b(curl|wget)\b.*(\.env|kaggle\.json|id_rsa|id_ed25519|metadata\.google|169\.254\.169\.254)", "Refusing
        (r"\b(psql|sqlite3)\b.*\b(drop\s+database|drop\s+schema)\b", "Refusing database/schema drop from the agent.
    ]
    for pattern, reason in deny_patterns:
        if re.search(pattern, lc, flags=re.IGNORECASE):
            return "deny", reason

    # Ask: irreversible or product/competition decision gates.
    ask_patterns = [
        (r"\brm\b", "File deletion requires approval unless implemented through an approved cleanup command."),
        (r"\brmdir\b", "Directory deletion requires approval."),
        (r"\bfind\b.*\s-delete\b", "find -delete requires approval."),
        (r"\bgit\s+clean\b", "git clean requires approval."),
        (r"\bgh\s+pr\s+merge\b", "Merging a PR is a decision gate."),
        (r"\bkaggle\s+competitions\s+submit\b", "Kaggle submission requires human approval."),
        (r"\bkaggle\s+kernels\s+push\b", "Kaggle kernel push requires human approval."),
        (r"\bkaggle\s+datasets\s+(create|version)\b", "Kaggle dataset publication requires human approval."),
        (r"\bpython\s+tools/kca/kca\.py\s+kaggle\s+(submit|push|upload)\b", "KCA Kaggle publication requires human
        (r"\bsupabase\s+db\s+(push|reset)\b", "Remote/destructive Supabase database operation requires approval."),
        (r"\b(psql|sqlite3)\b.*\b(truncate|delete\s+from)\b", "Destructive SQL requires approval."),
        (r"\bdocker\s+compose\s+down\b.*\s-v\b", "Docker volume deletion requires approval."),
    ]
    for pattern, reason in ask_patterns:
        if re.search(pattern, lc, flags=re.IGNORECASE):
            return "ask", reason
```

```
    return None, None


def main() -> int:
    try:
        payload = json.load(sys.stdin)
    except Exception as exc:
        emit("deny", f"Guard hook could not parse input: {exc}")
        return 0

    tool = payload.get("tool_name", "")
    tool_input = payload.get("tool_input") or {}

    if tool in {"Read", "Edit", "Write"}:
        file_path = tool_input.get("file_path") or tool_input.get("path") or ""
        if path_is_protected(file_path):
            emit("deny", f"Protected path blocked: {file_path}")
            return 0

    if tool == "Bash":
        command = tool_input.get("command") or ""
        decision, reason = bash_decision(command)
        if decision and reason:
            emit(decision, reason)
            return 0

    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

11. Product code competition agent 的权限策略

KCA 产品里的 agent 不应该简单复制 Claude Code 的 ask/deny，而应该更结构化：

```
Allow by default:
  - edit files inside job workspace
  - run tests
  - run benchmark
  - create artifacts
  - summarize logs
  - call LiteLLM/local models
  - create branch
  - commit changes
  - create PR

Decision gate:
  - submit to Kaggle
  - merge PR
  - publish dataset/kernel
  - apply remote/prod Supabase migration
  - spend above budget threshold
  - change model tier to expensive model
  - change competition direction/objective

Deny:
  - read secrets
  - write outside workspace
  - sudo/root host changes
  - rm outside workspace
  - drop database/schema
  - force push
  - network exfiltration
```

产品中建议把 permission decision 变成数据结构：

```
{
  "operation": "kaggle.submit",
  "default_decision": "ask",
  "reason": "External competition submission",
  "allowed_in_modes": ["human-approved", "competition-finalization"],
  "audit_required": true
}
```

这样 UI 可以显示：

```
Agent reached decision gate:
- Submit run kca-run-123 to Kaggle competition xyz?
- Current local CV: 0.8123
- Public LB estimate: unknown
- Artifact: submission.csv
- Cost so far: $1.42
Approve / Reject / Ask agent to improve
```

12. 我给你的最终建议

现在最合适的路线是：

```
开发 KCA:
  继续用 Claude Code

Product UI shell:
  继续在 Claude Code 里做 source of truth
  Lovable 只用于视觉原型/灵感/landing page mock

KCA CLI:
  commands = deterministic operator commands
  skills = agent workflow recipes
  tools = executable actions
  jobs = durable agent runs
  permissions = external policy, not prompt-only

Claude Code dev permissions:
  allow broad tools
  ask only irreversible/decision operations
  deny secrets and host-destructive commands
  keep audit hook
  modify guard_tool.py to stop asking on git push / gh pr create
  use --permission-mode dontAsk or auto for /goal tasks
```

可以让 Claude Code 下一步直接做这个任务：

```
Update .claude/settings.json and .claude/hooks/guard_tool.py to a permissive KCA development policy.

Goal:
Allow Claude Code to run milestone /goal tasks with minimal human approval, while still blocking secrets and host-d

Requirements:
- Allow normal coding, tests, lint, build, docker, git commit/push, gh pr create, local Supabase migration, and KCA
- Ask only for irreversible or decision-level operations:
  rm/rmdir/git clean, gh pr merge, Kaggle submit/upload/push, Supabase db push/reset, destructive SQL, Docker volum
- Deny secrets, .env reads/writes, SSH/AWS/GCloud/GitHub token reads, sudo/su/doas, force push, privileged docker,
- Keep audit_event.py hook.
- Do not put real secrets in config.
- Update docs explaining the permission policy.

Acceptance criteria:
- `claude --permission-mode dontAsk` can run tests/build/git/gh pr create without approval.
- Secrets are still blocked.
- Destructive operations produce deny or ask.
- Existing audit log still records Bash/Edit/Write events.
```